NTU DSAI Module 2 Capstone

Building an end-to-end data pipeline and analysis flow for a company

Quick overview of use case

Who?	• An SME which has, in recent years, accumulated substantial surplus free cash resources in its fiscal reserves.
Why?	• The CFO has been tasked by the Board to evaluate opportunities to invest into the US tech equities which have skyrocketed due to the rise of AI. • The Board members have identified 5 tech stocks for the SME's investment appraisal and requires a Board presentation and recommendation on the portfolio allocation strategy at the next bi-annual Board meeting in Q3 of CY2026.
What?	• As the supporting analyst team, you have been tasked by the CFO to gather market data to facilitate the Treasury Team's appraisal of the market risk implications of investing in such highly volatile equity counters. • The end-goal is to enable the CFO to make a risk-aware and well-considered portfolio proposal for the Board's approval in Q3.

Project deliverables & key considerations

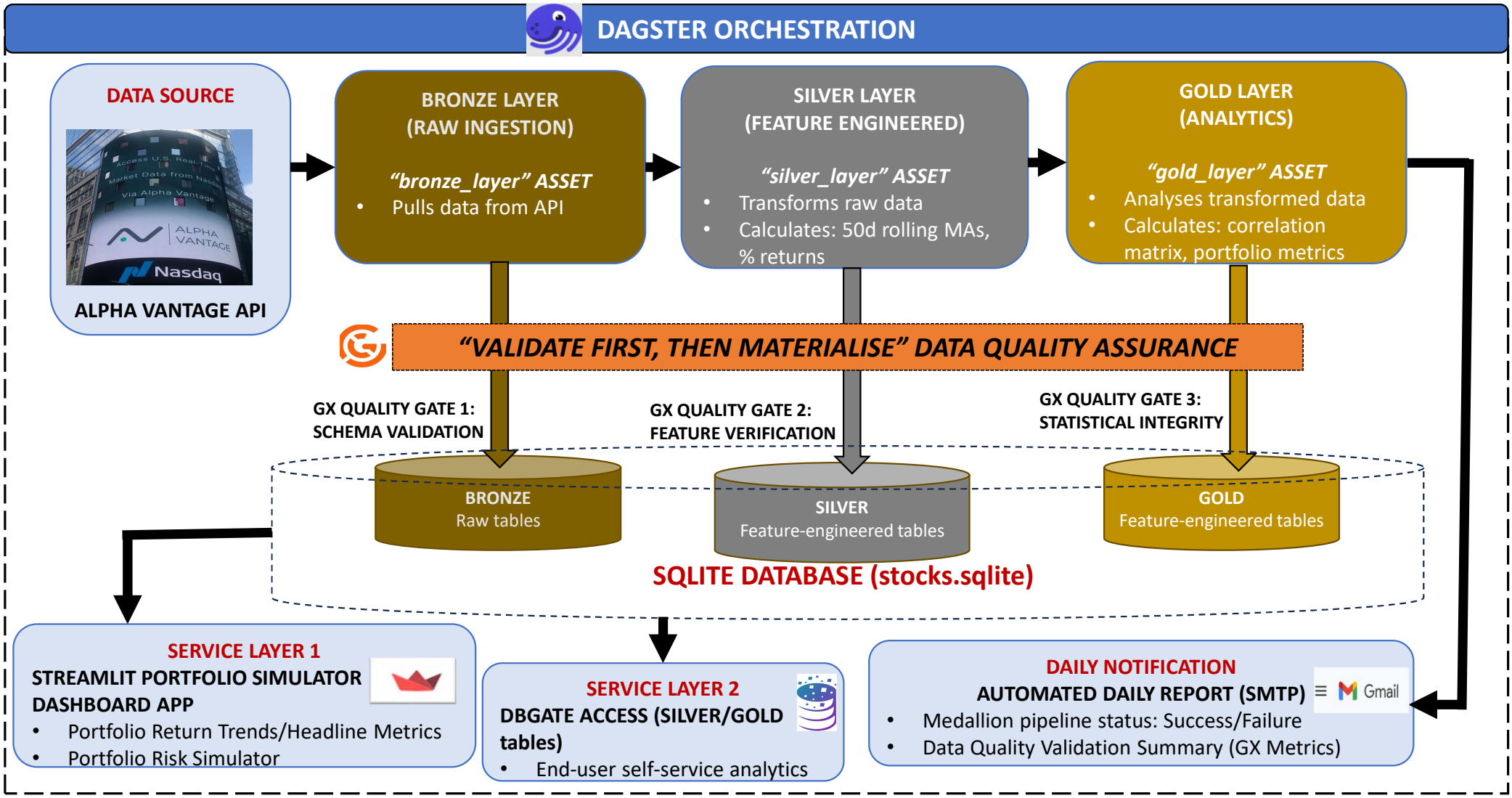
Deliverables

- An efficient end-to-end, hassle-free data pipeline for timely ingestion of stock price data into a locally hosted database for Treasury Team's convenient internal access for analysis and monitoring.
- A dashboard app to facilitate simulations of various portfolio allocation strategies and analysis of portfolio risk posture.

Considerations

- Currency, accuracy and reliability of stock price data.
- Minimal operational & maintenance overheads.
- No additional acquisition of physical IT assets/infrastructure or expensive subscriptions to financial databases like Bloomberg.

The end-to-end data pipeline solution



Pipeline design choices

Why Medallion architecture?

Transparency & Lineage
- Clear data refinement stages & visibility.
- Traceability of pipeline glitches for efficient troubleshooting.

Data Governance
- Incremental "validate first, then materialise" at each sequential layer evidences robust data governance controls & mitigates "Garbage In-Garbage Out" risks for the Treasury Team & CFO.

↓

Achieves currency, accuracy & reliability project considerations!

Why SQLite + Dagster + DbGate + Streamlit data stack?

SQLite
- Open-source, light-weight, portable & tested RDB solution with zero setup, licensing & admin overheads.

Dagster
- Open-source, versatile, built-in asset lineage & developer-friendly data orchestration tool for modern data stacks with free core.

DbGate
- Free open-source, cross-platform SQL & NoSQL DB manager supporting wide range of DBs, user-friendly data browser, editor & query functionalities

Streamlit
- Free open-source Python-driven, minimal front-end expertise requirement for rapid interactive dashboard app building.

↓

Achieves minimal operational overheads & zero IT asset acquisition project considerations!

Medallion-layered GX quality gates & test design

GX QUALITY GATE 1

SCHEMA VALIDATION

```
configs = [
    ExpectationConfiguration(
        expectation_type="expect_column_values_to_not_be_null",
        kwargs={"column": "close"}
    ),
    ExpectationConfiguration(
        expectation_type="expect_column_values_to_be_between",
        kwargs={"column": "volume", "min_value": 0}
    )
]
```

1. As “close” is the price column for computing stock returns, any null values in the time series would upset metric calculations/accuracy, and must be rectified before ingestion into stocks.sqlite Bronze layer.
2. Negative “volume” numbers – if any – signals anomalous behaviour, and must be rectified before ingestion into stocks.sqlite Bronze layer.

GX QUALITY GATE 2

FEATURE VERIFICATION

```
expected_features = ["return_pct", "rolling_50"]
failed_tickers = []
for t in settings.tickers:
    try:
        raw_df = pd.read_sql(f"SELECT * FROM BRONZE_{t}", conn)
        silver_df = process_rolling_metrics(raw_df)

        configs = [
            ExpectationConfiguration(
                expectation_type="expect_column_to_exist",
                kwargs={"column": feature}
            ) for feature in expected_features
        ]
```

1. The “return_pct” and “rolling_50” (aka stock daily returns and 50-day rolling averages) columns are specifically feature-engineered into the Silver layer to augment the Bronze layer metadata.
2. As these features are key to stock analytics/trend-spotting for data-driven investment decisions, it is critical to verify their successful insertions into the Silver tables.

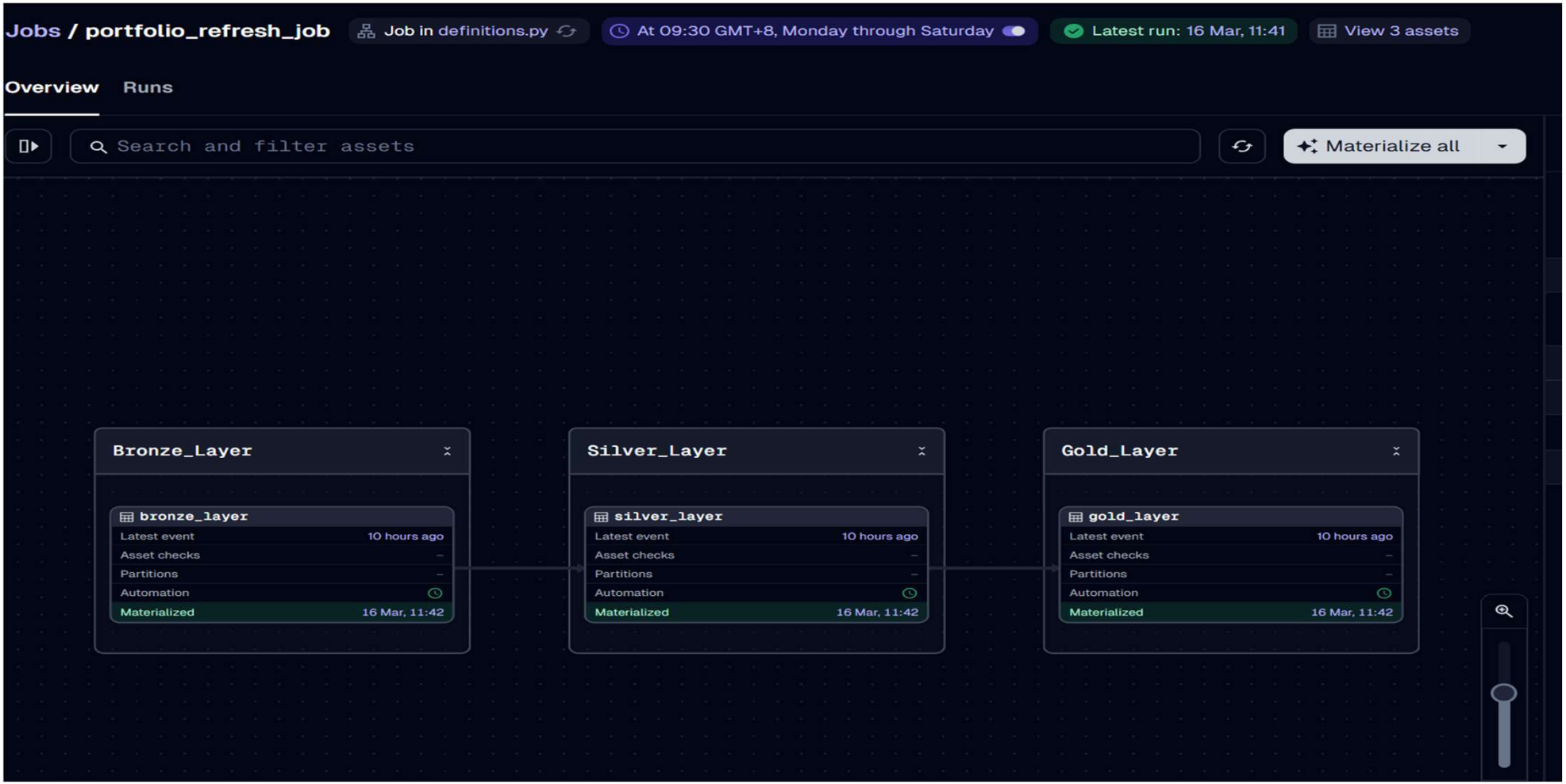
GX QUALITY GATE 3

STATISTICAL INTEGRITY

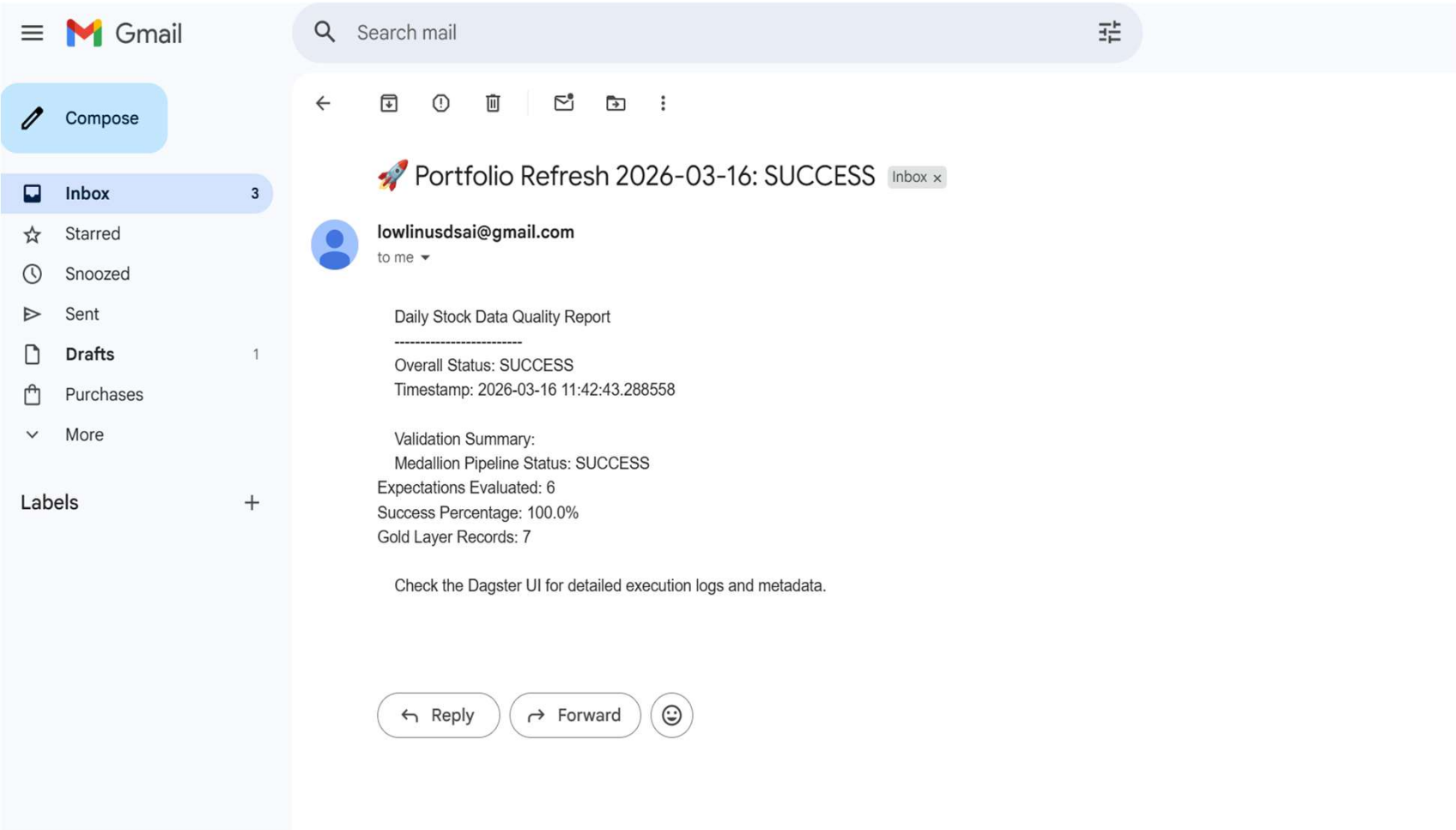
```
configs = [
    ExpectationConfiguration(
        expectation_type="expect_column_values_to_be_between",
        kwargs={"column": c, "min_value": -1.0001, "max_value": 1.0001}
    ) for c in corr_matrix.columns
]
```

1. The “corr_matrix” object reports the stock returns correlation matrix; this is a critical statistical indicator of portfolio diversification/risk reduction benefits in the Markowitz risk-return analysis.
2. Given that correlation numbers could influence portfolio asset allocation strategy, it is critical to ensure statistical integrity [i.e. that the pairwise correlations are well within the (-1,1) limit].

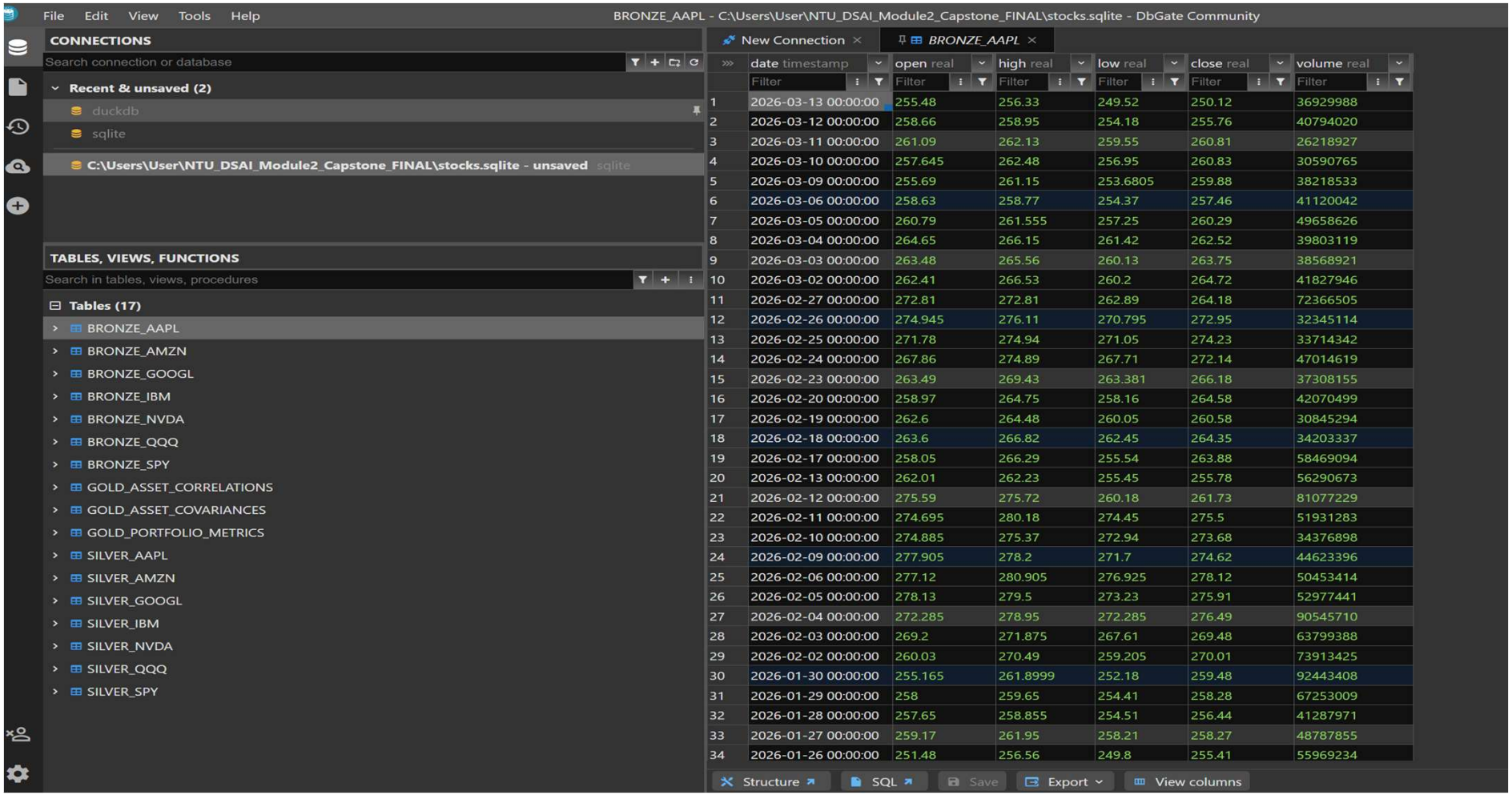
Pipeline snapshots: Dagster orchestration asset graph



Pipeline snapshots: Daily email notifications

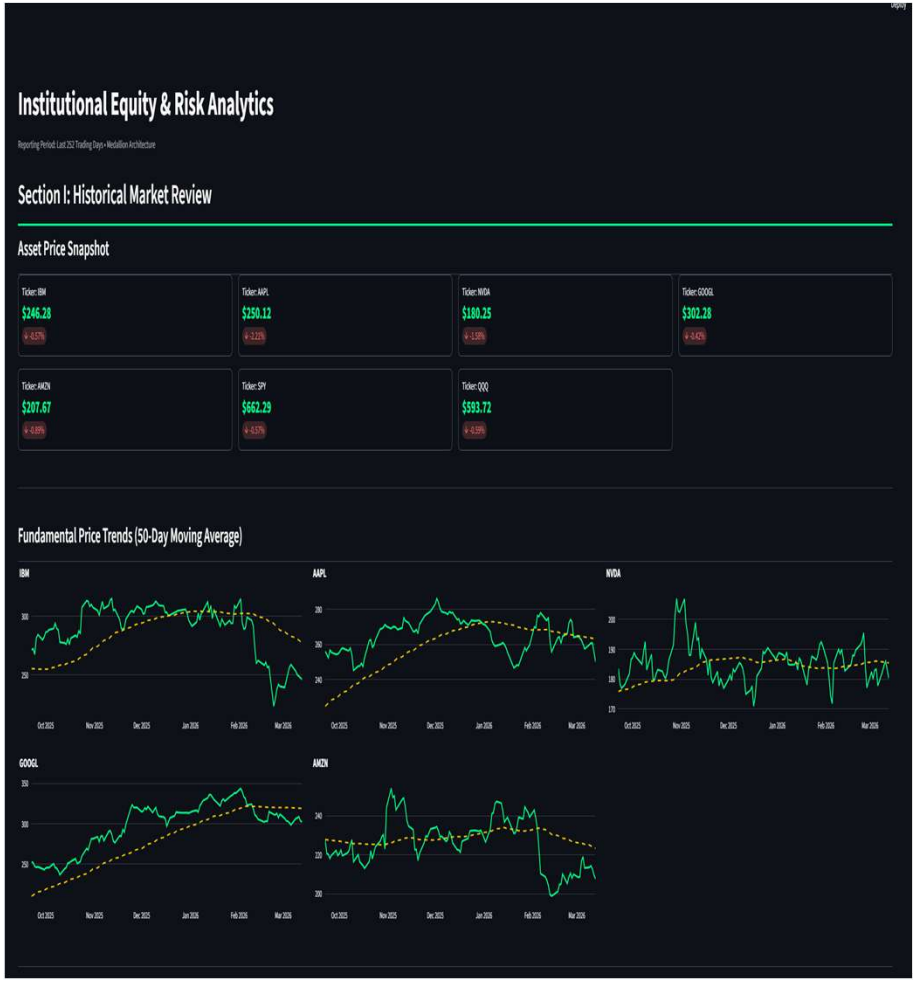


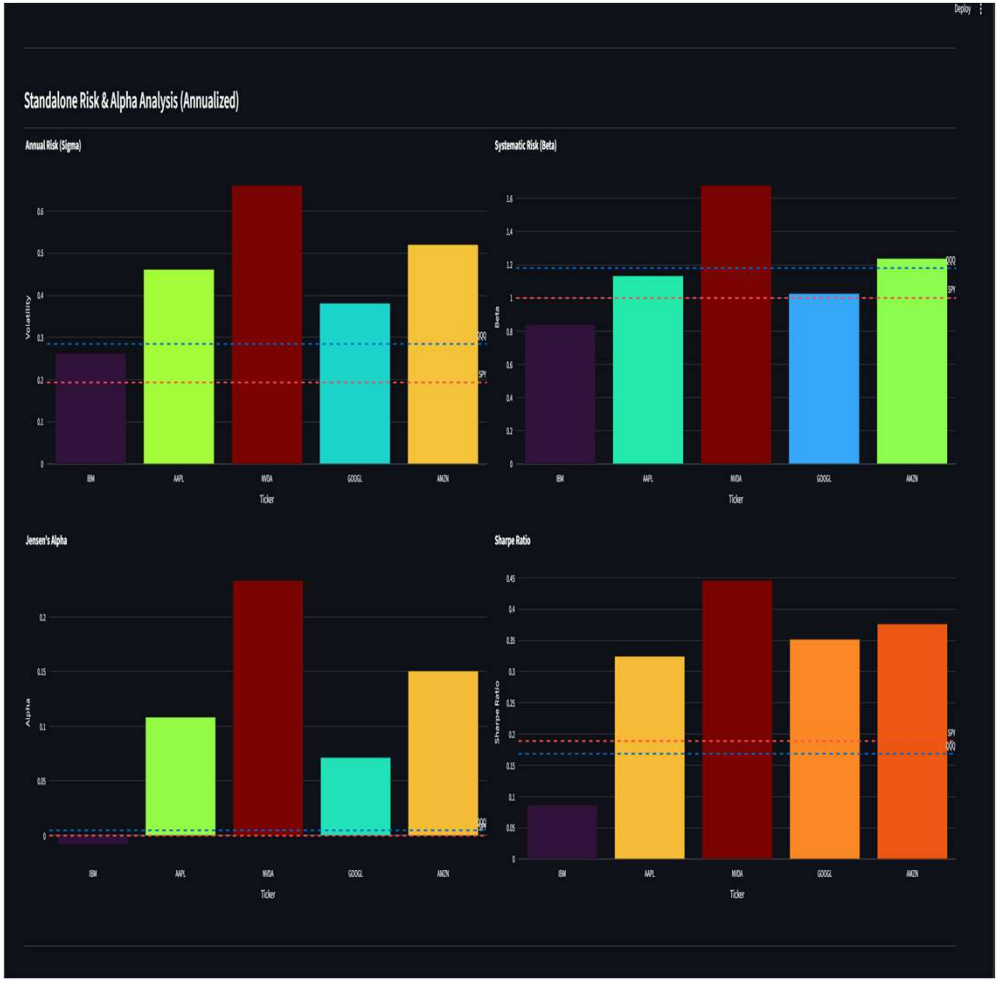
Pipeline snapshots: SQLite database in DbGate



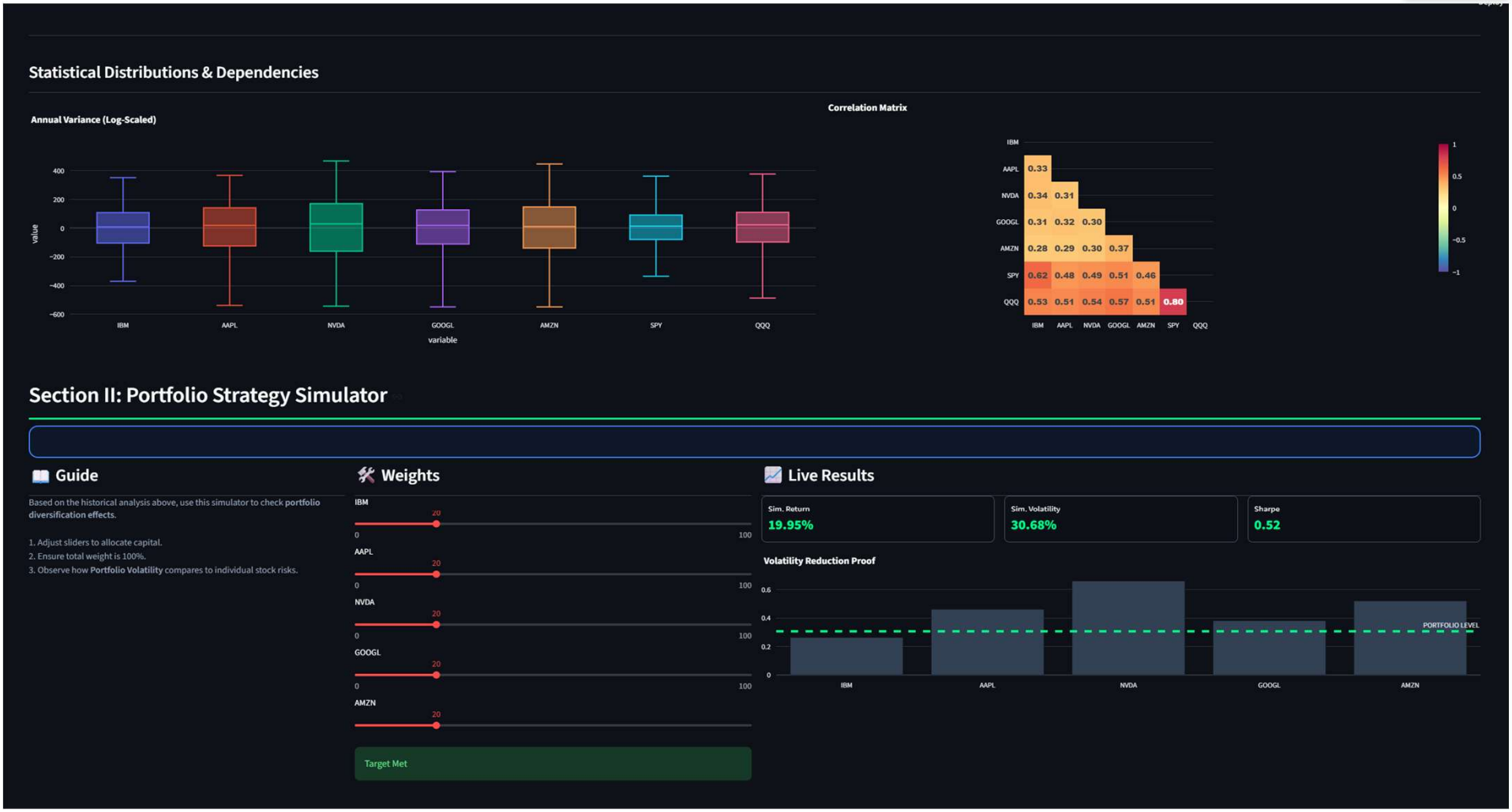
	date timestamp	open real	high real	low real	close real	volume real
1	2026-03-13 00:00:00	255.48	256.33	249.52	250.12	36929988
2	2026-03-12 00:00:00	258.66	258.95	254.18	255.76	40794020
3	2026-03-11 00:00:00	261.09	262.13	259.55	260.81	26218927
4	2026-03-10 00:00:00	257.645	262.48	256.95	260.83	30590765
5	2026-03-09 00:00:00	255.69	261.15	253.6805	259.88	38218533
6	2026-03-06 00:00:00	258.63	258.77	254.37	257.46	41120042
7	2026-03-05 00:00:00	260.79	261.555	257.25	260.29	49658626
8	2026-03-04 00:00:00	264.65	266.15	261.42	262.52	39803119
9	2026-03-03 00:00:00	263.48	265.56	260.13	263.75	38568921
10	2026-03-02 00:00:00	262.41	266.53	260.2	264.72	41827946
11	2026-02-27 00:00:00	272.81	272.81	262.89	264.18	72366505
12	2026-02-26 00:00:00	274.945	276.11	270.795	272.95	32345114
13	2026-02-25 00:00:00	271.78	274.94	271.05	274.23	33714342
14	2026-02-24 00:00:00	267.86	274.89	267.71	272.14	47014619
15	2026-02-23 00:00:00	263.49	269.43	263.381	266.18	37308155
16	2026-02-20 00:00:00	258.97	264.75	258.16	264.58	42070499
17	2026-02-19 00:00:00	262.6	264.48	260.05	260.58	30845294
18	2026-02-18 00:00:00	263.6	266.82	262.45	264.35	34203337
19	2026-02-17 00:00:00	258.05	266.29	255.54	263.88	58469094
20	2026-02-13 00:00:00	262.01	262.23	255.45	255.78	56290673
21	2026-02-12 00:00:00	275.59	275.72	260.18	261.73	81077229
22	2026-02-11 00:00:00	274.695	280.18	274.45	275.5	51931283
23	2026-02-10 00:00:00	274.885	275.37	272.94	273.68	34376898
24	2026-02-09 00:00:00	277.905	278.2	271.7	274.62	44623396
25	2026-02-06 00:00:00	277.12	280.905	276.925	278.12	50453414
26	2026-02-05 00:00:00	278.13	279.5	273.23	275.91	52977441
27	2026-02-04 00:00:00	272.285	278.95	272.285	276.49	90545710
28	2026-02-03 00:00:00	269.2	271.875	267.61	269.48	63799388
29	2026-02-02 00:00:00	260.03	270.49	259.205	270.01	73913425
30	2026-01-30 00:00:00	255.165	261.8999	252.18	259.48	92443408
31	2026-01-29 00:00:00	258	259.65	254.41	258.28	67253009
32	2026-01-28 00:00:00	257.65	258.855	254.51	256.44	41287971
33	2026-01-27 00:00:00	259.17	261.95	258.21	258.27	48787855
34	2026-01-26 00:00:00	251.48	256.56	249.8	255.41	55969234

Pipeline snapshots: Streamlit dashboard app [1/2]





Pipeline snapshots: Streamlit dashboard app [2/2]



Project code files & related collaterals are accessible via my GitHub repository at the following URL:

https://github.com/lowlinus178/NTU_DSAI_Module2_Capstone